Neural Networks 206 (2027) 109810

Contents lists available at ScienceDirect

Neural Networks

journal homepage: www.elsevier.com/locate/neunet

Full Length Article

FedRSFL: Federated recommender systems with feature-level user data sharing

Chaoqun Yang [a], Quoc Viet Hung Nguyen [a], Hongzhi Yin [b], Liang Qu [c,*]

[a] Griffith University, Gold Coast, QLD, Australia
[b] The University of Queensland, Brisbane, QLD, Australia
[c] Edith Cowan University, Perth, WA, Australia

ARTICLE INFO

Keywords:
Recommender systems
Federated recommender systems
User data sharing

ABSTRACT

Federated recommender systems have made great progress in recent years, and they can protect the privacy of user data while maintaining recommendation performance. However, most existing federated recommender systems require every user to keep all data strictly on the local device, and they overlook the users who care less about privacy and are willing to share data with the server for better recommendation. To address this, a few recent studies explore federated recommender systems with user data sharing, which allow users to share their data with the server. However, these methods mainly focus on interaction-level user-item data, which is often sparse, and they overlook the finer-grained feature-level data. To address this problem, this paper explores a federated recommender system framework with feature-level user data sharing, termed FedRSFL, which allows users to share part of their features with the server. However, training on the shared data is non-trivial: the same shared interaction produces two inconsistent training signals, because a client learns it from the complete local data with its private user embedding while the server learns it from the shared part alone with a separate user embedding, and directly aggregating the two models mixes unaligned parameters and hurts the global model. FedRSFL therefore couples the two sides through a bidirectional preference distillation, in which the clients teach the server the complete preference that the shared data misses, while the server teaches the clients the cross-user pattern that a single client cannot see, so that the shared data improves recommendation and every user embedding stays on its device. Experiments on three public datasets demonstrate that feature-level user data sharing improves the recommendation accuracy, with an average gain of about 3.8% in HR@10 over the strongest baseline on each dataset.

1. Introduction

Recommender systems have become a core part of many online services, and they now support everyday experiences such as short-video feeds and e-commerce product suggestions (Chen et al., 2025; Huang et al., 2026). A common way to build such a system is to learn user preferences from the interaction history between users and items, and then rank candidate items by the predicted preference (He et al., 2017). A well-known difficulty is that this interaction data is often long-tailed and very sparse, so most users and items have only a few records. A typical remedy is to bring in side information, such as user features, to ease the data sparsity (Rendle, 2010). However, collecting user data and user features in one central place creates a large privacy risk and can conflict with data protection regulations such as the GDPR (Fig. 1(a)) (European Parliament and Council of the European Union, 2016).

In recent years, federated learning has made promising progress in protecting privacy while keeping good model utility (McMahan et al., 2017). Its core idea is to keep each user's data on the local device and train the model locally, so that a user only uploads model gradients, and a central server aggregates these gradients into a global model and sends the global model back to the clients (Fig. 1(b)) (McMahan et al., 2017). Building on this idea, many studies bring federated learning into recommendation and form federated recommender systems (FedRec), which aim to address the privacy concerns of recommender systems (Ammad-ud din et al., 2019). Representative methods range from the early FCF (Ammad-ud din et al., 2019) to FedNCF (Perifanis & Efraimidis, 2022), and further to personalized methods such as PFedRec (Zhang et al., 2023). However, most existing federated recommender systems assume that every user has a very high privacy demand and is unwilling to share any data, while in practice some users care less about privacy and would

* Corresponding author.
E-mail address: l.qu@ecu.edu.au (L. Qu).

https://doi.org/10.1016/j.neunet.2026.109810
Received 5 July 2026; Received in revised form 6 October 2026; Accepted 26 October 2026
Available online 29 October 2026
0893-6080/© 2026 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

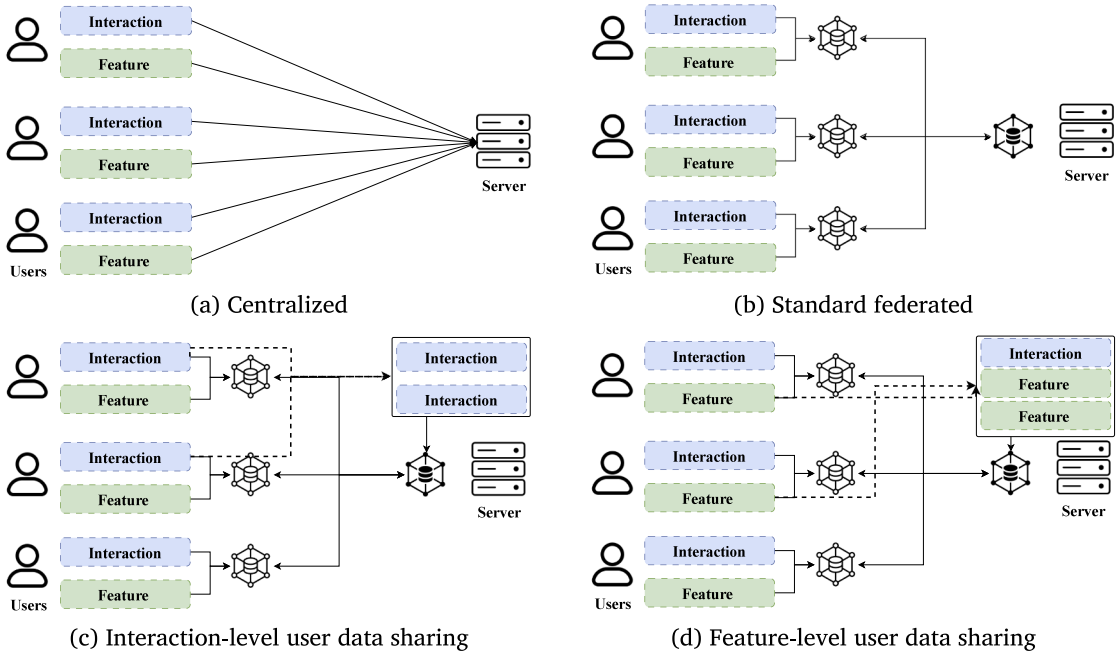


Fig. 1. Four settings for recommendation. (a) Centralized recommendation gathers all interaction history and user features on a server. (b) Standard federated recommendation keeps data on the device and shares only gradients. (c) Federated recommender systems with interaction-level user data sharing let users voluntarily share part of their interaction data. (d) Federated recommender systems with feature-level user data sharing let users voluntarily share part of their user features on top of the interaction channel, which is the setting of this paper.

share data in exchange for better recommendation, and current designs overlook this fact.

To address this issue, several recent studies start to explore federated recommender systems in which users share data on a voluntary basis. A first group shares data at the interaction level, which we refer to as federated recommender systems with interaction-level user data sharing (Fig. 1(c)). The main idea is that each user is free to decide whether and how much of their interaction data to share with the server, so that the server can access more data and reduce the performance drop that the federated setting suffers from non-independent and identically distributed data. For example, User-Governed Data Contribution lets each user control whether and how much interaction data to contribute to the server (Qu et al., 2024), and PDC-FRS shares perturbed interaction data under differential privacy to train an auxiliary model that enriches each user's local data (Yang et al., 2024). A limitation of these methods is that they only support coarse-grained user-item interaction data, but as noted above the interaction data is often sparse and sometimes needs side information such as user features to help.

Beyond the interaction channel, users may also share data at the feature level, which we refer to as federated recommender systems with feature-level user data sharing (Fig. 1(d)). This channel does not replace interaction-level sharing but adds to it, and in our setting a user shares feature fields only on top of shared interactions. Building on this feature-level setting, it is non-trivial to make good use of the shared data. A natural idea is to let the server train a recommender directly on the shared data, with its own user embedding table for the sharing users. However, this creates a deeper challenge: the same shared interaction now produces two inconsistent training signals. On the device, the interaction is learned against the complete local data of the user, and the private user embedding absorbs everything the user did not share. On the server, the same interaction is learned against only the shared part of the user with a separate user embedding, and different users share different amounts of interactions and different feature fields, so the server-side view of each user is partial and varies from user to user. As a result, the two sides embed the same user in two spaces that are never aligned, and directly aggregating the two models, for example through federated averaging, mixes parameters that were learned against unaligned user representations and hurts the global model. The question is therefore how to let the two sides learn from each other without mixing their unaligned parameters, so that the sharing users obtain the better recommendation they expect.

In this paper, we introduce a federated recommender system framework with feature-level user data sharing, termed FedRSFL, which aims to make the shared data effectively improve the overall model performance, especially for users who contribute shared data. In FedRSFL, users first share part of their interactions and, on top of that, part of their feature fields, and the server collects them into a server-side dataset. The clients and the server side then train the same base recommender on their own data, where the server holds the shared data and trains on it with its own user embedding table for the sharing users. This reveals a complementary structure between the two sides. A normal client holds all of its own local data but sees only itself, so it lacks the collaborative signal across users. The server holds shared data from many users and can learn its own user embeddings, but this data is incomplete, so its signal is only a partial view of user preference. FedRSFL therefore refines the two sides through a client-server bidirectional preference distillation. Because the two sides use different user embedding spaces, they teach each other at the level of predictions rather than parameters: the clients pass to the server the complete preference that the shared data misses, and the server passes back the pattern across users that a single client cannot learn from its own data alone. Finally, the client models and the server-side model are aggregated into the global model, and the uploaded updates are protected by local differential privacy.

We summarize our contributions as follows.

- We study an underexplored research problem, namely federated recommender systems that support user data sharing at the finer-grained feature level, where a user who already shares interactions may further share feature fields of their own choice to ease the data sparsity and obtain better recommendation.
- We propose a federated recommender system framework with feature-level user data sharing, termed FedRSFL. At its core is

a client-server bidirectional preference distillation, in which the clients teach the server-side model the complete preference that the shared data misses, and the server-side model teaches back the pattern across users that a single device cannot see.
- We conduct extensive experiments on three public datasets. FedRSFL outperforms both the non-sharing federated recommender systems and the recent sharing methods, with an average gain of about 3.8% in HR@10 over the strongest baseline, and the group analysis shows that the sharing users obtain the better recommendation they expect while the non-sharing users are not harmed.

The rest of this paper is organized as follows. Section 2 reviews related work. Section 3 gives the preliminaries and the problem formulation. Section 4 presents the proposed FedRSFL. Section 5 reports the experiments, and Section 6 concludes the paper.

2. Related work

This section reviews the two lines of work that are most related to ours. The first line is federated recommender systems, and the second line is federated recommender systems with user data sharing, in which users voluntarily share their interaction data or user features.

2.1. Federated recommender systems

Federated recommender systems (FedRec) are motivated by the privacy concerns of recommender systems, and their core idea is to keep each user's data on the local device and train the model locally, so that a user only uploads model gradients and a central server aggregates these gradients into a global model. Along this line, FCF (Ammad-ud din et al., 2019) first adapts collaborative filtering to the federated setting by keeping user factors on the device and updating item factors on the server. FedMF (Chai et al., 2021) studies matrix factorization in this setting and adds encryption to the uploaded gradients for stronger privacy. MetaMF (Lin et al., 2020) uses a meta network on the server to generate private rating prediction models for devices with little data. FedFast (Muhammad et al., 2020) speeds up training through a client sampling and aggregation strategy that goes beyond simple averaging. FedNCF (Perifanis & Efraimidis, 2022) brings neural collaborative filtering into the federated setting so that the model can capture non-linear user-item relations. FedGNN (Wu et al., 2021) trains a graph neural network on each device and expands the local user-item graph to use higher-order signals. PFedRec (Zhang et al., 2023) learns a personalized model for each user through a dual personalization mechanism on both the user side and the item side. GPFedRec (Zhang et al., 2024) builds a user graph from the similarity of item embeddings and uses a graph-guided aggregation to personalize the global model. KGFedRS (Ma et al., 2025) brings knowledge graphs into the federated setting and uses their auxiliary information to ease the data sparsity while protecting both the users and the knowledge graph. FedRAP (Li et al., 2024) learns personalized local user embeddings and a global item view that is added to a private item view on each device for prediction.

However, all of these methods assume that every user has a very high privacy demand and is unwilling to share any data, so a user only uploads model gradients and never shares data with the server. This assumption overlooks the users who care less about privacy and would share data in exchange for better recommendation.

2.2. Federated recommender systems with user data sharing

To use the signal from such willing users, several recent studies explore federated recommender systems in which each user voluntarily decides whether to share data and how much to share, which we refer to as federated recommender systems with user data sharing. Depending on what a user shares, these methods perform interaction-level or feature-level user data sharing. FPL (Anelli et al., 2021) lets each user control how much preference data leaves the device while a central factorization model is trained with a learning-to-rank objective. UC-FedRec (Hu & Song, 2024) moves from interaction data to user features and lets each user set per-attribute privacy preferences, and it learns filters that remove the information of the attributes a user wants to protect. User-Governed Data Contribution (Qu et al., 2024) allows each user to decide whether and how large a proportion of the interaction data to contribute, so that the server can access more data and reduce the performance loss from non-independent and identically distributed data. PDC-FRS (Yang et al., 2024) shares perturbed interaction data under differential privacy and trains an auxiliary model that enriches each user's local data with global collaborative information.

However, most of these methods only support coarse-grained user-item interaction data, and the interaction data is often sparse and sometimes needs side information such as user features to help. UC-FedRec is the only work that reaches the feature level, but it treats the shared user features as a liability to protect rather than a resource to use, which leaves the collaborative value of voluntarily shared user features unexploited.

3. Preliminaries

This section provides the basic definitions and notation used throughout this work, and then formulates the problem of federated recommender systems with feature-level user data sharing. In this setting, each user trains on a local device, and a user may voluntarily share part of their interactions and, on top of that, part of their user features with the server, so that the server can use the shared data to improve recommendation. Throughout the paper, we use calligraphic letters for sets, uppercase letters for matrices, bold lowercase letters for vectors, and lowercase letters for scalars. Table 1 summarizes the main notation.

- **Users (clients):** the set of users is $\mathcal{U} = \{1, \dots, N\}$, where N is the number of users. Each user $u \in \mathcal{U}$ holds a local device and acts as a client in the federated protocol.
- **Items:** the set of items is $\mathcal{I} = \{1, \dots, M\}$, where M is the number of items.
- **User-item interaction data:** the implicit feedback is a matrix $R \in \{0,1\}^{N \times M}$, where $r_{ui} = 1$ if user u has interacted with item i and $r_{ui} = 0$ otherwise. We write $\mathcal{I}_u = \{i \in \mathcal{I} : r_{ui} = 1\}$ for the items that user u has interacted with, which resides on the device of user u.
- **User features and feature fields:** the user features are organized into T feature fields. Each field $t \in \{1, \dots, T\}$ is a categorical attribute, such as gender, age, or occupation, and it takes a value from a finite set $\mathcal{X}_t = \{1, \dots, C_t\}$, where $C_t = |\mathcal{X}_t|$ is the number of values of field t (for example, the gender field has $C_t = 2$). User u takes the value $x_{u,t} \in \mathcal{X}_t$ on field t, so its feature vector is $\mathbf{x}_u = (x_{u,1}, \dots, x_{u,T}) \in \mathcal{X}$, where $\mathcal{X} = \mathcal{X}_1 \times \cdots \times \mathcal{X}_T$ is the feature space shared by all users. The feature matrix $X \in \mathcal{X}^N$ stacks these vectors as its rows.
- **Local model:** each user u keeps a local model on the device, parameterized by θ_u, which scores candidate items for that user and predicts a preference $\hat{r}_{ui}$.
- **Server and global model:** S denotes the single central server that coordinates the federated training. It maintains a global model, parameterized by θ_g, which is shared across users and updated by aggregating the updates from the clients.

3.1. Problem formulation

We now formulate federated recommender systems with feature-level user data sharing. This setting builds on interaction-level user data sharing and does not replace it, so a user may voluntarily share data through two channels. Through the interaction channel, each user u may share a subset $\mathcal{I}_u^s \subseteq \mathcal{I}_u$ of its interactions, as in prior work. Through the feature channel, which is the focus of this paper, each user decides on

Table 1
Main notation used throughout the paper.

Symbol	Description
$\mathcal{U}$, N	set of users and number of users
$\mathcal{I}$, M	set of items and number of items
R, r_{ui}	interaction matrix and its entry
$\mathcal{I}_u$	items that user u has interacted with
$\mathcal{I}_u^s$	interactions that user u voluntarily shares
$\mathcal{U}_S$	set of sharing users, $\{u : \mathcal{I}_u^s \neq \emptyset\}$
$\mathcal{D}_u^s$	data that user u shares
T	number of feature fields
C_t	number of values of feature field t
$\mathcal{X}_t$, $\mathcal{X}$	value set of field t and the feature space
$\mathbf{x}_u$, X	feature vector of user u and the feature matrix
O, $o_{u,t}$	feature-sharing matrix and its entry
$\mathcal{D}_S$	shared data observed by the server
$\mathcal{S}$	central server
d	embedding dimension
$\mathbf{p}_u$, $\tilde{\mathbf{p}}_u$	user embedding on the client and on the server
θ_g	shared parameters $\{Q, E, \Phi\}$ (item, feature, MLP)
$\mathbf{e}(\mathbf{x}_u)$	feature embedding of user u
$\hat{r}_{ui}$	predicted preference of user u for item i
$\mathcal{B}$	clients selected in the current round
$\mathcal{C}_u$	candidate item set for distillation
$\mathbf{s}_u^c$, $\mathbf{s}_u^s$	client and server soft labels on $\mathcal{C}_u$
α, β	distillation weights of the two directions
τ	temperature of the distillation
w_u	distillation weight of user u
λ	aggregation weight of the server
ε, C	privacy budget and clipping norm

their own which feature fields to share, described by a sharing matrix $O \in \{0,1\}^{N\times T}$, where $o_{u,t} = 1$ if user u shares the value of field t and $o_{u,t} = 0$ otherwise. The rows of O can differ from user to user, which means that different users may share different feature fields, and a user may share nothing at all. In this paper the feature channel is layered on the interaction channel, so a user may share feature fields only in addition to sharing some interactions, and sharing features without any shared interaction is not allowed, because the server then has no way to place the user in the shared model. Formally, $o_{u,t} = 1$ implies $\mathcal{I}_u^s \neq \emptyset$. We write $\mathcal{U}_S = \{u \in \mathcal{U} : \mathcal{I}_u^s \neq \emptyset\}$ for the set of sharing users, which are exactly the users the server can represent.

Putting the two channels together, the server gathers everything the users share into the shared data $\mathcal{D}_S$, whose construction we give in Section 4.1, while the rest of the interaction data and the withheld feature values stay on the local devices. The shared features form the part that this paper turns into a useful signal.

The goal is to raise the recommendation performance in this setting. Given the local interaction data $\{\mathcal{I}_u\}_{u\in\mathcal{U}}$ and the voluntarily shared user features, we learn the global model θ_g and the local models $\{\theta_u\}_{u\in\mathcal{U}}$ by

$$\min_{\theta_g, \{\theta_u\}} \sum_{u\in\mathcal{U}} \mathcal{L}_u\big(\theta_g, \theta_u;\ \mathcal{I}_u,\ \{x_{u,t} : o_{u,t} = 1\}\big), \tag{1}$$

where $\mathcal{L}_u$ is the local recommendation loss of user u. A better solution gives each user a top-K item list of higher quality, which we measure by ranking metrics such as HR@K and NDCG@K. Two constraints hold throughout. First, only model updates and the data that a user voluntarily shares reach the server, and the rest stays on the local device. Second, sharing is voluntary and heterogeneous, so each user u first chooses whether and how much interaction data to share, and, on top of that, their own feature-sharing pattern $\mathbf{o}_u = (o_{u,1}, \ldots, o_{u,T})$.

4. Proposed method: FedRSFL

To let existing federated recommender systems give better recommendations to the users who care less about privacy and would share finer-grained feature data, this paper introduces a federated recommender system framework with feature-level user data sharing, termed FedRSFL. As shown in Fig. 2, the proposed method has four stages. The first stage is **user data sharing**, where users share their interaction data and side information, such as user features in this work, with the server, so that the server builds a dataset from the shared data. The second stage is **client-side and server-side training**, where the clients that join a round of federated training train a local model on their own data, and the server trains a server-side model on the shared data at the same time. The third stage is **client-server bidirectional preference distillation**, where, for the users who shared their data, we build a distillation set from their shared interactions and let the client side and the server side distill from each other, so that the shared data refines the model for these sharing users. The fourth stage is **global aggregation**, where federated averaging combines the server-side parameters with the parameters of the clients in the current round into the global model, which is then sent out for the next round. Algorithm 1 gives the detailed steps.

4.1. User data sharing

Traditional federated recommender systems assume that every user has a very high privacy demand, so they keep all data strictly on the local device (Ammad-ud din et al., 2019; Perifanis & Efraimidis, 2022). This design overlooks the users who care less about privacy and would share part of their data with the server in exchange for better recommendation. A few recent methods support such voluntary sharing, but they only support user-item data at the interaction level (Qu et al., 2024; Yang et al., 2024). However, the interaction data of recommender systems is very sparse, and a common remedy is to bring in side information to ease this problem, such as user features (Rendle, 2010). FedRSFL therefore lets a user share data at the feature level as well.

We now describe how the users share their data. In FedRSFL the feature channel is layered on the interaction channel. A user u first decides whether to share interactions, and a sharing user reveals a subset $\mathcal{I}_u^s \subseteq \mathcal{I}_u$ of the local interactions. Only on top of that may the user further share feature fields, described by the sharing pattern $\mathbf{o}_u$ from Section 3. We do not support sharing features without sharing any interaction, because such features cannot be trained on the server side: without a single interaction from user u, the server has no preference signal for this user, so it has no way to connect the shared feature values to any preference. Formally, $o_{u,t} = 1$ implies $\mathcal{I}_u^s \neq \emptyset$, and the sharing users form the set $\mathcal{U}_S = \{u \in \mathcal{U} : \mathcal{I}_u^s \neq \emptyset\}$. The data that user u shares is

$$\mathcal{D}_u^s = \{(u, i) : i \in \mathcal{I}_u^s\} \cup \{(u, t, x_{u,t}) : o_{u,t} = 1\}, \tag{2}$$

and the server collects the shared data of all sharing users into one server-side dataset,

$$\mathcal{D}_S = \bigcup_{u\in\mathcal{U}_S} \mathcal{D}_u^s. \tag{3}$$

Compared with the local dataset of a single client, which contains the data of one user only, $\mathcal{D}_S$ pools the interactions and features of many users, so it carries an explicit collaborative signal across users that the server can learn from directly.

4.2. Client-side and server-side training

After the users have shared their data, FedRSFL trains with a standard federated procedure, in which the client side and the server side train the same base recommender on their own data.

4.2.1. Client-side training

In each round, the server first randomly selects a set $\mathcal{B} \subseteq \mathcal{U}$ of clients to participate in the current round of federated training, and each selected client trains on its own device. We adopt the NeuMF architecture (He et al., 2017) as the base recommendation model for all clients and for the server side, because it captures both the linear and the non-linear

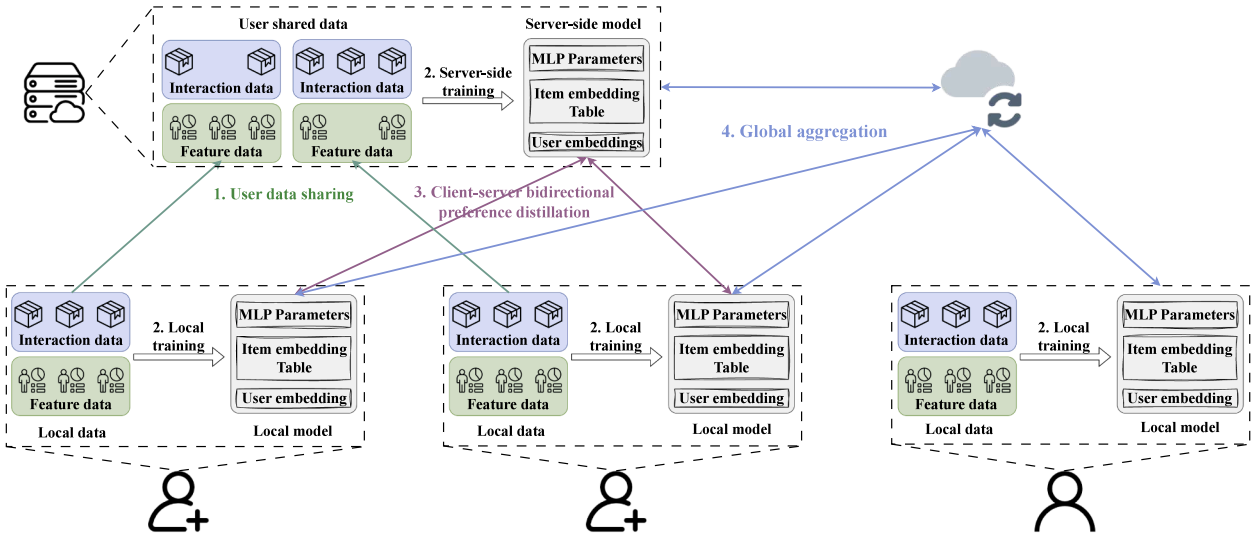


Fig. 2. Overview of FedRSFL. Users share part of their interactions and features with the server, the clients and the server side train the same base recommender on their own data, the two sides refine each other through a client-server bidirectional preference distillation, and the client models and the server-side model are aggregated into the global model.

part of the user-item relation.[1] We now describe the forward pass of the model on a client. Given a user u and an item i from the local dataset, the model first maps the user id and the item id into embeddings of dimension d through the user embedding tables and the item embedding tables,

$$\begin{aligned} \mathbf{p}_u^G &= P^G[u], & \mathbf{p}_u^M &= P^M[u], \\ \mathbf{q}_i^G &= Q^G[i], & \mathbf{q}_i^M &= Q^M[i], \end{aligned} \tag{4}$$

where P^G and P^M are the user embedding tables of the two branches of NeuMF, and Q^G and Q^M are the corresponding item embedding tables, collected in Q. The user features are mapped in the same way: each feature field t has its own feature embedding table $E_t \in \mathbb{R}^{C_t \times d}$, the value $x_{u,t}$ of user u selects one row of this table, and the rows of all fields are concatenated into the feature embedding of the user,

$$\mathbf{e}(\mathbf{x}_u) = \left[E_1[x_{u,1}]; \dots ; E_T[x_{u,T}] \right]. \tag{5}$$

The generalized matrix factorization branch of NeuMF models the linear relation between the user and the item by an element-wise product of their embeddings,

$$\mathbf{h}^G = \mathbf{p}_u^G \odot \mathbf{q}_i^G. \tag{6}$$

The multilayer perceptron branch models the non-linear relation, and this is where the features enter: the user embedding is concatenated with the feature embedding and the item embedding, and the result is fed into the multilayer perceptron,

$$\mathbf{h}^M = \mathrm{MLP}\left(\left[\mathbf{p}_u^M ; \mathbf{q}_i^M ; \mathbf{e}(\mathbf{x}_u) \right] \right). \tag{7}$$

The outputs of the two branches are then concatenated and fed into the prediction layer, which gives the preference score

$$\hat{r}_{ui} = \sigma\left(\mathbf{w}^\top \left[\mathbf{h}^G ; \mathbf{h}^M \right] \right), \tag{8}$$

where σ is the sigmoid function, and $\mathbf{w}$ together with the multilayer perceptron weights are collected in Φ. Because the feedback is implicit, the client trains with the Bayesian personalized ranking loss (Rendle et al., 2009), so that an observed item is scored above an unobserved one. For a user u, a positive item $i \in \mathcal{I}_u$, and a sampled negative item $j \notin \mathcal{I}_u$, the loss is

$$\mathcal{L}_{\mathrm{BPR}} = -\sum_{(u,i,j)} \ln \sigma\left(\hat{r}_{ui} - \hat{r}_{uj} \right), \tag{9}$$

and minimizing this loss on the complete local interactions ends the client-side training of the round.

4.2.2. Server-side training

The server side trains in the same way as a client: it runs the same NeuMF model with the same ranking loss, but on the server-side dataset $\mathcal{D}_S$ built in Section 4.1. We refer to this server-side recommender as the server. Two points differ from a normal client. First, the server joins the training in every round, while the normal clients are sampled per round, because $\mathcal{D}_S$ pools the shared data of many users and thus carries a larger and more stable signal. Second, the server does not reuse the user embeddings of the clients. It maintains its own user embedding table $\tilde{\mathbf{p}}$ for the sharing users $\mathcal{U}_S$. The reason is that even for a sharing user, the user embedding on the device is never uploaded, because this embedding also encodes the interactions and the feature values that the user chose not to share, and uploading it would leak exactly the data the user wants to keep. For the features, the server sees only the shared fields of a user, and a field that is not shared contributes a zero row to the feature embedding.

4.3. Client-server bidirectional preference distillation

After the client-side and server-side training, the two sides hold two models that were trained on different views of the data, and the two views are complementary. A client trains on the complete data of one user, so its prediction reflects the full preference of that user, but it never sees any other user. The server trains on the shared data of many users, so its prediction reflects the pattern across users, but for each user it sees only the shared part. Moreover, the two sides represent a user in different spaces, since a client uses $\mathbf{p}_u$ while the server uses $\tilde{\mathbf{p}}_u$, so their knowledge cannot be exchanged through the user embeddings. We therefore let the two sides teach each other at the level of predictions, which does not depend on the user embedding space. The insight is that each side passes on exactly what the other side cannot learn alone: the client teaches the server the complete preference that the shared data misses, and the server teaches the client the pattern across users that a single device cannot see, so both sides improve.

[1] FedRSFL is model-agnostic, because its four stages only use the scores of the base model. Therefore NeuMF can be replaced by other base recommenders, such as graph-based models.

The distillation is built on the shared interaction data. It runs only for the users who shared their interactions and who also participate in the current round, that is, $u \in \mathcal{U}_S \cap \mathcal{B}$, because only for these users does the same signal appear on the client side and on the server side within the round. For each such user we build a candidate item set C_u from the shared interactions $\mathcal{I}_u^s$ together with a set of sampled negative items. The negative items are sampled uniformly from the items that the user has not interacted with, and C_u is built once before training and kept fixed, so that both sides score the same items in every round. Both sides score every item in C_u and turn the scores into a soft preference through a softmax with a temperature τ, which gives the client soft label $\mathbf{s}_u^c$ and the server soft label $\mathbf{s}_u^s$,

$$\mathbf{s}_u^c = \mathrm{softmax}\big(\hat{\mathbf{r}}_u^c/\tau\big), \qquad \mathbf{s}_u^s = \mathrm{softmax}\big(\hat{\mathbf{r}}_u^s/\tau\big), \tag{10}$$

where $\hat{\mathbf{r}}_u^c$ and $\hat{\mathbf{r}}_u^s$ are the scores of the two sides on C_u. The distillation then runs in two directions. In the first direction the client is the teacher: the client soft label reflects the complete preference, so the server learns to reproduce it from the shared data, which repairs its partial view. In the second direction the server is the teacher: the server soft label reflects the pattern across users, so the client uses it as an extra signal, with a weight w_u that is larger for users with few interactions, since these users gain the most from the shared signal. The server and a participating client therefore minimize

$$\mathcal{L}^s = \mathcal{L}^s_{\mathrm{BPR}} + \alpha \sum_{u \in \mathcal{U}_S \cap \mathcal{B}} \mathrm{KL}\big(\mathbf{s}_u^c \,\|\, \mathbf{s}_u^s\big), \tag{11}$$

$$\mathcal{L}^u = \mathcal{L}^u_{\mathrm{BPR}} + \beta\, w_u\, \mathbf{1}[u \in \mathcal{U}_S]\, \mathrm{KL}\big(\mathbf{s}_u^s \,\|\, \mathbf{s}_u^c\big), \tag{12}$$

where α and β balance the ranking loss and the distillation, and the indicator $\mathbf{1}[u \in \mathcal{U}_S]$ keeps the second term only for the sharing clients, so a non-sharing client minimizes the ranking loss alone. Minimizing $\mathcal{L}^s$ updates the server-side model, namely $\tilde{\mathbf{p}}$ together with the server copy of the embedding tables and the perceptron, and minimizing $\mathcal{L}^u$ updates the client-side model, namely $\mathbf{p}_u$ together with the client copy, so the distillation refines the model parameters of both sides for the sharing users.

Algorithm 1 FedRSFL: one communication round.

Require: global shared parameters θ_g, shared data $\mathcal{D}_S$, server soft labels $\{\mathbf{s}_u^s\}$
1: the server selects the clients $\mathcal{B}$ of this round and sends them θ_g; it sends the soft label $\mathbf{s}_u^s$ only to each sharing client $u \in \mathcal{U}_S \cap \mathcal{B}$
2: **for** each client $u \in \mathcal{B}$ **in parallel do**
3: update $\{\mathbf{p}_u, \theta_g^u\}$ by minimizing $\mathcal{L}^u$ in Eq. (12) on the local data, where the distillation term uses $\mathbf{s}_u^s$ and is active only if $u \in \mathcal{U}_S$
4: **if** $u \in \mathcal{U}_S$ **then**
5: compute the soft labels $\mathbf{s}_u^c$ on C_u by Eq. (10)
6: **end if**
7: protect the update of θ_g^u by Eqs. (13)–(15) and upload it with $\mathbf{s}_u^c$
8: **end for**
9: the server updates $\{\tilde{\mathbf{p}}, \theta_g^s\}$ by minimizing $\mathcal{L}^s$ in Eq. (11) on $\mathcal{D}_S$
10: refresh the server soft labels $\{\mathbf{s}_u^s\}$ by Eq. (10)
11: aggregate $\{\theta_g^u\}_{u \in \mathcal{B}}$ and θ_g^s into θ_g by Eq. (16)
12: distribute θ_g to the clients of the next round and to the server
Ensure: updated θ_g and $\{\mathbf{s}_u^s\}$

4.4. Global aggregation

After the distillation, FedRSFL updates the global model. Each client that participated in the current round uploads its locally updated parameters, which include the item embedding tables Q, the multilayer perceptron together with the prediction layer Φ, and the feature embedding tables $E = \{E_1, \ldots, E_T\}$. We collect these shared parameters as $\theta_g = \{Q, E, \Phi\}$. The user embeddings $\mathbf{p}_u$ and $\tilde{\mathbf{p}}$ are private and never leave their owner.

Uploading these parameters is still not free of risk, even though no raw data is uploaded. During local training, only the rows of the items that the user interacted with, and only the rows of the feature values that the user holds, receive non-zero gradients, so a curious server can read the non-zero rows of an uploaded update of Q or E and recover which items the user consumed and which feature values the user has Chai et al. (2021). To address this problem, FedRSFL integrates local differential privacy into the upload, following the practice of clipping and noising model updates in differentially private federated learning (McMahan et al., 2018). Let $\mathbf{\Delta}_u$ denote the update of the shared parameters that client u produced in the current round. The update is first clipped to a bounded L_2 norm C, so that no single client can move the global model by more than C,

$$\tilde{\mathbf{\Delta}}_u = \mathbf{\Delta}_u \cdot \min\Big(1,\ \frac{C}{\|\mathbf{\Delta}_u\|_2}\Big). \tag{13}$$

The clipped update is then perturbed with Gaussian noise through the Gaussian mechanism (Dwork & Roth, 2014),

$$\hat{\mathbf{\Delta}}_u = \tilde{\mathbf{\Delta}}_u + \mathcal{N}\big(0, \sigma^2 \mathbf{I}\big), \qquad \sigma = \frac{C\sqrt{2\ln(1.25/\delta)}}{\varepsilon\, |\mathcal{B}|}, \tag{14}$$

where ε is the privacy budget, $\delta = 10^{-5}$, and the noise scale shrinks with the number of participating clients $|\mathcal{B}|$, because the noise of the averaged update is what matters for utility. The client then uploads the protected parameters

$$\theta_g^u = \theta_g + \hat{\mathbf{\Delta}}_u, \tag{15}$$

so a smaller ε gives stronger privacy at the cost of more noise, and the averaged update stays usable while each upload is protected. We study the effect of this protection in Section 5.7.

The server then aggregates the uploaded parameters with federated averaging (McMahan et al., 2017), where the participants of the aggregation are the clients of the current round together with the server,

$$\theta_g \leftarrow (1-\lambda)\, \frac{1}{|\mathcal{B}|} \sum_{u \in \mathcal{B}} \theta_g^u + \lambda\, \theta_g^s, \tag{16}$$

where θ_g^u is the uploaded parameter of client u, θ_g^s is the parameter of the server, and the weight λ controls how much the server contributes, so that the server, which holds a large pool of shared data, does not dominate the global model. Finally, the aggregated parameters θ_g are distributed in the next round to the newly selected clients and to the server, and the next round begins. Algorithm 1 summarizes the whole procedure.

4.5. Communication and computation cost

We briefly analyze the cost of FedRSFL over a standard federated recommender system that uses the same base model. Let $|\mathcal{I}|$ be the number of items, d the embedding dimension, $\sum_t C_t$ the total number of feature values, and $|\Phi|$ the number of parameters of the multilayer perceptron and the prediction layer. In each round, a participating client downloads and uploads the shared parameters $\theta_g = \{Q, E, \Phi\}$, whose size is $O(|\mathcal{I}|d + d\sum_t C_t + |\Phi|)$, and this part is the same as in FedNCF. The only extra message is the soft label $\mathbf{s}_u^c$ uploaded by a sharing client and the soft label $\mathbf{s}_u^s$ sent down to it, and each of them has $O(|C_u|)$ numbers. The communication of a round is therefore

$$O\Big(|\mathcal{B}|\big(|\mathcal{I}|d + d\textstyle\sum_t C_t + |\Phi|\big) + |\mathcal{U}_S \cap \mathcal{B}|\,|C_u|\Big), \tag{17}$$

where the second term is negligible next to the first, because $|C_u|$ is a small constant that does not grow with the model or the item set, while $|\mathcal{I}|d$ alone is already far larger. For the computation, let F be the cost of one forward pass of the base model on a user-item pair, let n_u be the number of local training interactions of user u, and let K be the number of sampled negatives per positive. The ranking loss of a client costs $O(n_u K F)$ per round. The distillation adds one forward pass over the candidate set and the KL term, which costs $O(|C_u|F)$, and it applies only to

Table 2
Statistics of the datasets.

Dataset	#Users	#Items	#Interactions	#Fields
ML-100K	943	1,682	100,000	3
ML-1M	6,040	3,706	1,000,209	3
BlackFriday	5,891	3,631	550,068	6

the sharing clients, so a non-sharing client has no extra computation. The server-side training on $\mathcal{D}_S$ costs $O(|\mathcal{D}_S|KF)$ plus $O(|\mathcal{U}_S \cap \mathcal{B}|\,|C_u|F)$ for the distillation, which is the main additional computation of FedRSFL, but it runs on the server, which has far more resources than a client, and it adds no communication.

5. Experiments

We evaluate FedRSFL to answer five research questions.

- **RQ1**: how does FedRSFL perform against traditional federated recommender systems and sharing methods?
- **RQ2**: how does each main component of FedRSFL affect its performance?
- **RQ3**: how do the main hyperparameters affect its performance?
- **RQ4**: how does local differential privacy affect its performance?
- **RQ5**: how does the shared data affect the different user groups?

5.1. Datasets

We use three public datasets that come with user features. Table 2 lists their statistics.

- **ML-100K** (Harper & Konstan, 2015) contains movie ratings from users, where each user carries three feature fields, namely gender, age, and occupation. We treat a rating as a positive interaction and bin the age into groups as released with the data.
- **ML-1M** (Harper & Konstan, 2015) is a larger movie-rating dataset with the same three feature fields. We use the same preprocessing as for ML-100K.
- **BlackFriday** (Analytics Vidhya, 2018) records purchases in a retail setting, where each user carries six feature fields, namely gender, age, occupation, city category, years in the current city, and marital status. We treat a purchase as a positive interaction.

5.2. Baselines

We compare FedRSFL with five baselines. The first three are federated recommender systems without data sharing, and the last two share data.

- **FedMF** (Chai et al., 2021) is a federated matrix factorization method that keeps user factors on the device and protects the uploaded gradients.
- **FedNCF** (Perifanis & Efraimidis, 2022) brings neural collaborative filtering into the federated setting.
- **PFedRec** (Zhang et al., 2023) learns a personalized model for each user through a dual personalization mechanism.
- **PDC-FRS** (Yang et al., 2024) shares perturbed interaction data and trains an auxiliary model that enriches each user's local data.
- **UC-FedRec** (Hu & Song, 2024) lets each user share user features under per-attribute privacy preferences.

5.3. Experimental setup

For each user we split the interactions into training, validation, and test sets with a ratio of $7:2:1$. To simulate feature-level user data sharing, we divide the users into three groups. By default, 10% of the users are full-sharing and reveal all of their interactions and all of their feature fields, 30% are partial-sharing and reveal half of their interactions and half of their feature fields, and the remaining 60% share nothing. We later vary the two proportions of the partial-sharing users in the hyperparameter analysis. For the hyperparameters of FedRSFL, we select each value from a search range on the validation set. The embedding dimension d is chosen from $\{8, 16, 32, 64, 128\}$ and set to 32. The learning rate is chosen from $\{10^{-3}, 10^{-2}\}$ and set to 10^{-3}. The aggregation weight λ of the server is chosen from $\{0, 0.1, 0.3, 0.5, 0.7, 0.9\}$ and set to 0.1. The distillation weights α and β are chosen from $\{0.3, 0.5, 1, 2\}$ and $\{0.1, 0.3, 0.6\}$ and set to $\alpha = 0.5$ and $\beta = 0.1$, and the temperature of the distillation is $\tau = 2$. The candidate set C_u contains 20 items from the shared interactions of the user and 100 sampled negative items. The weight for the users with few interactions is $w_u = \min(\max(\sqrt{\bar{n}/n_u}, 0.5), 3)$, where n_u is the number of local interactions of user u and $\bar{n}$ is the average over all users. The privacy budget of the local differential privacy is $\varepsilon = \infty$ in the main results, that is, no noise is added, and is varied in $\{0.1, 0.5, 1, 2, 4, 8\}$ in Section 5.7. The baselines use their standard settings provided by the authors. Following common practice in federated recommendation, we evaluate with the sampled ranking protocol, where each held-out item is ranked against 99 items that the user has not interacted with. We report HR@10 and NDCG@10, averaged over all test users and three independent runs.

Table 3
Overall performance under the default sharing setting. The best result in each column is in bold.

Method	ML-100K		ML-1M		BlackFriday	
	HR@10	NDCG@10	HR@10	NDCG@10	HR@10	NDCG@10
FedMF	0.741	0.461	0.752	0.476	0.666	0.385
FedNCF	0.734	0.458	0.763	0.496	0.667	0.390
PFedRec	0.727	0.451	0.759	0.493	0.669	0.392
PDC-FRS	0.733	0.462	0.762	0.490	0.669	0.391
UC-FedRec	0.737	0.458	0.761	0.489	0.679	0.398
FedRSFL	**0.795**	**0.518**	**0.777**	**0.505**	**0.694**	**0.413**

5.4. Top-K recommendation (RQ1)

This experiment compares FedRSFL with the baselines on the three datasets under the default sharing setting. In top-K recommendation, a model ranks the items that a user has not interacted with and returns the K items with the highest predicted preference, and a method is better if the held-out test items of the user appear higher in this list. Table 3 reports HR@10 and NDCG@10. We observe the following.

- FedRSFL achieves the best HR@10 and NDCG@10 on all three datasets. Averaged over the three datasets, it improves HR@10 by about 3.8% and NDCG@10 by about 6% over the strongest baseline of each dataset. This shows that the shared feature-level data, once turned into a usable signal by the server and the distillation, improves the overall recommendation accuracy.
- The methods that use shared data outperform the methods that do not. Comparing the two families, the average HR@10 of the three sharing methods, namely PDC-FRS, UC-FedRec, and FedRSFL, is higher than that of the three non-sharing methods on every dataset (0.755 against 0.734 on ML-100K, 0.767 against 0.758 on ML-1M, and 0.681 against 0.667 on BlackFriday). The reason is that the voluntarily shared data gives the server a direct training signal beyond the model updates, which the non-sharing methods can never access.
- The advantage holds on the feature-rich BlackFriday as well, where each user has six feature fields and FedRSFL reaches 0.694 HR@10 against 0.679 for the strongest baseline. A richer feature set gives the server more to learn from the shared data.

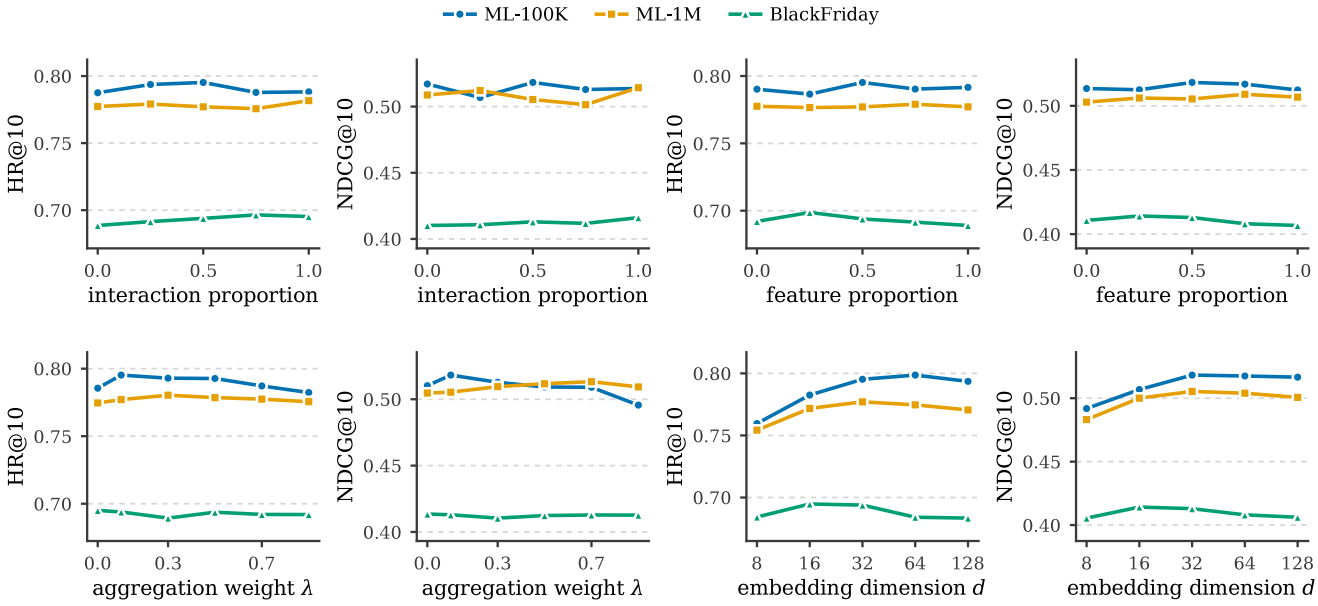


Fig. 3. Hyperparameter analysis of FedRSFL. The first row varies the interaction and feature sharing proportions of the partial-sharing users, and the second row varies the aggregation weight λ of the server and the embedding dimension d.

5.5. Ablation study (RQ2)

This experiment studies three main components of FedRSFL. We remove the bidirectional preference distillation (*w/o Distill*), the server-side training on the shared data (*w/o Server*), and the larger distillation weight for sparse users (*w/o Cold*). Fig. 4 reports HR@10 and NDCG@10 on ML-100K. We observe the following.

- The full model performs best on both metrics, so every component contributes to the final accuracy.
- Removing the distillation causes the largest drop in HR@10. Without it the two sides train in isolation, so the knowledge that the server learns from the shared data is not passed back to the clients, and the shared data only helps the server-side model itself.
- Removing the server-side training causes the largest drop in NDCG@10. Without it the shared data is never used in training, so the feature tables and the cross-user signal are lost.
- Removing the larger weight for the users with few interactions causes only a small drop, which shows that steering the distillation toward these users is helpful but is a secondary factor.

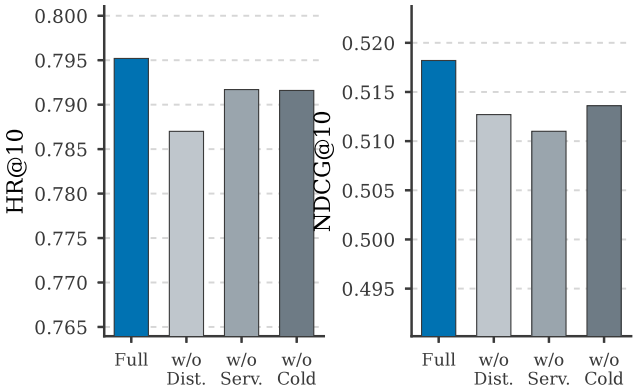


Fig. 4. Ablation study of FedRSFL on ML-100K.

5.6. Hyperparameter analysis (RQ3)

This experiment studies three hyperparameters, namely the interaction and feature sharing proportions of the partial-sharing users, the aggregation weight λ of the server, and the embedding dimension d. We vary the two sharing proportions in $\{0, 0.25, 0.5, 0.75, 1\}$, where one proportion is varied and the other stays at its default of 0.5. We vary λ in $\{0, 0.1, 0.3, 0.5, 0.7, 0.9\}$, where $\lambda = 0$ removes the server from the aggregation. We vary d in $\{8, 16, 32, 64, 128\}$. Fig. 3 reports the results on the three datasets. We observe the following.

- The accuracy is nearly unchanged when the partial-sharing users share more or fewer of their interactions or feature fields, and the curves show no clear trend on any dataset. The reason is that the server learns mainly from the full-sharing users, and a partial-sharing user already enters the server-side training once a part of the data is shared, so a larger part from the same user adds little new signal. This means that the method does not require the users to share much, and a small share is already enough to obtain the benefit.
- A moderate aggregation weight works best. On the two MovieLens datasets the accuracy rises from $\lambda = 0$, where the server is removed from the global model and the shared knowledge is lost, to a moderate value, and a large λ then hurts, because the server, which sees only the shared part of each user, overwrites the global model. BlackFriday is insensitive to λ. We keep the value selected on the validation set, and the results show that a range of moderate values gives a similar accuracy.
- The accuracy rises with the embedding dimension d and then flattens, so we use a moderate dimension as a balance between accuracy and cost. A larger dimension brings no further gain and slightly overfits on the two larger datasets.

5.7. Effect of local differential privacy (RQ4)

FedRSFL protects the uploaded parameters with local differential privacy. In each round we clip the aggregated update of the shared parameters to an L_2 norm C and add Gaussian noise once through the

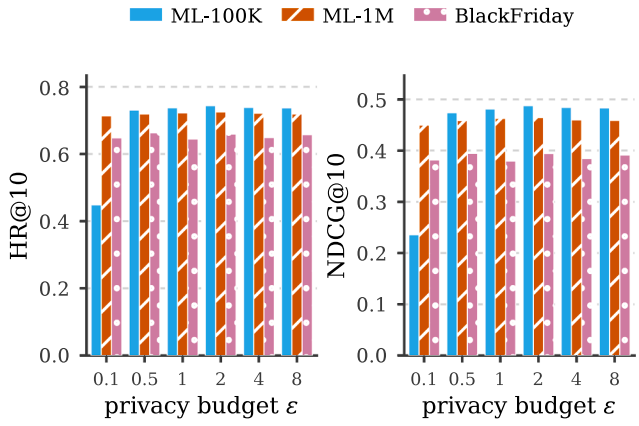


Fig. 5. Effect of the privacy budget ε of local differential privacy.

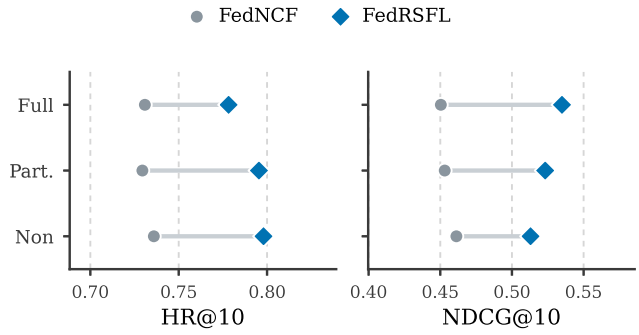


Fig. 6. Performance by user group on ML-100K, comparing the FedNCF baseline with FedRSFL (Part. and Non denote the partial-sharing and non-sharing users).

Gaussian mechanism with $\delta = 10^{-5}$, where the noise scale is calibrated to the number of participating clients. We study how the strength of the privacy affects the utility by varying the privacy budget ε, where a smaller ε adds more noise and gives stronger privacy. Fig. 5 reports HR@10 and NDCG@10 on the three datasets. We observe the following.

- Under every finite budget the accuracy is somewhat below the results without noise in Table 3, and this cost stays almost the same as the budget shrinks over most of the range and is similar on the three datasets. The reason is that the cost comes mainly from clipping the aggregated update, which is applied with the same norm under every finite budget, while the noise that is added once per round is too small to matter in this range and does not accumulate over rounds.
- The noise itself matters only when the budget is very small and the number of participating clients is small, because the noise of the averaged update shrinks as more clients contribute. The two larger datasets therefore stay nearly flat over the whole range of budgets, while the small ML-100K loses clearly more accuracy at the strongest privacy level, yet it still stays far above random ranking.

5.8. User group analysis (RQ5)

In FedRSFL, a user who shares data expects better recommendation in return. At the same time, the sharing should not introduce a bias that hurts the other groups, and in particular the users who share nothing should not become worse. We therefore group the test users into full-sharing, partial-sharing, and non-sharing users, and we compare FedRSFL with the FedNCF baseline on each group. Fig. 6 reports HR@10 and NDCG@10 for each group on ML-100K. We observe the following.

- The sharing users obtain better recommendation under FedRSFL than under the non-sharing baseline, and the users who share more tend to gain more. This is the return that the sharing users expect from sharing their data, since the server learns their preference from the shared data and passes it back through the distillation.
- The non-sharing users are not harmed and also benefit from the sharing of others, although their gain is smaller than that of the sharing users. The reason is that the knowledge the server learns from the shared data enters the global model through the aggregation and is thus distributed to every client, so a user who values privacy can keep everything local and still benefit from the data shared by others.

6. Conclusion

We studied federated recommender systems with feature-level user data sharing, where users voluntarily share part of their features with the server on top of the interaction channel. We showed that the shared data is hard to use directly, because the preference of a user is carried by the user embedding that stays on the device and also encodes the data the user did not share. We proposed FedRSFL, which lets the server train on the shared data with its own user embedding table, and trains it together with the normal clients through a bidirectional preference distillation that runs alongside federated averaging. Experiments on three public datasets show that FedRSFL outperforms both non-sharing federated recommender systems and recent methods that share only interaction data, and that the sharing users obtain the better recommendation they expect while the non-sharing users are not harmed. In future work, we plan to extend FedRSFL to a dynamic setting, where the data of a user changes over time and the user may change what to share at any moment, so the shared data on the server side also changes over time, and the model needs to follow these changes.

CRediT authorship contribution statement

Chaoqun Yang: Writing – original draft, Methodology, Formal analysis, Conceptualization; **Quoc Viet Hung Nguyen:** Writing – review & editing, Conceptualization; **Hongzhi Yin:** Writing – review & editing, Supervision, Formal analysis; **Liang Qu:** Writing – review & editing, Validation, Conceptualization.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the authors used ChatGPT (OpenAI) for language editing, including checking grammatical errors and improving the clarity and readability of the manuscript. The authors reviewed and edited the output as needed and take full responsibility for the content of the published article.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

Ammad-ud din, M., Ivannikova, E., Khan, S. A., Oyomno, W., Fu, Q., Tan, K. E., & Flanagan, A. (2019). Federated collaborative filtering for privacy-preserving personalized recommendation system. arXiv:1901.09888.

Analytics Vidhya (2018). Black friday dataset. https://www.kaggle.com/datasets/sdolezel/black-friday.

Anelli, V. W., Deldjoo, Y., Di Noia, T., Ferrara, A., & Narducci, F. (2021). How to put users in control of their data in federated top-n recommendation with learning to rank. In *Proceedings of the 36th annual ACM symposium on applied computing (SAC)*. https://doi.org/10.1145/3412841.3442010

Chai, D., Wang, L., Chen, K., & Yang, Q. (2021). Secure federated matrix factorization. *IEEE Intelligent Systems*, *36*(5), 11–20.

Chen, R., Fan, J., Wu, M., & Ma, S. (2025). Conditional diffusion model for recommender systems. *Neural Networks*, *185*, 107204. https://doi.org/10.1016/j.neunet.2025.107204

Dwork, C., & Roth, A. (2014). The algorithmic foundations of differential privacy. *Foundations and Trends in Theoretical Computer Science*, *9*(3–4), 211–407. https://doi.org/10.1561/0400000042

European Parliament and Council of the European Union (2016). Regulation (EU) 2016/679 of the European parliament and of the council (General data protection regulation). Official Journal of the European Union, L119.

Harper, F. M., & Konstan, J. A. (2015). The MovieLens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems (TiiS)*, *5*(4), 19:1–19:19. https://doi.org/10.1145/2827872

He, X., Liao, L., Zhang, H., Nie, L., Hu, X., & Chua, T.-S. (2017). Neural collaborative filtering. In *Proceedings of the 26th international conference on world wide web (WWW)* (pp. 173–182). https://doi.org/10.1145/3038912.3052569

Hu, Q., & Song, Y. (2024). User consented federated recommender system against personalized attribute inference attack. In *Proceedings of the 17th ACM international conference on web search and data mining (WSDM)*. https://doi.org/10.1145/3616855.3635830

Huang, Z., Zhang, S., Cheng, D., Li, J., Liu, L., Lu, G., & Zhang, G. (2026). Learning instrumental variable representation for debiasing in recommender systems. *Neural Networks*, *193*, 107977. https://doi.org/10.1016/j.neunet.2025.107977

Li, Z., Long, G., & Zhou, T. (2024). Federated recommendation with additive personalization. In *International conference on learning representations* (pp. 11770–11787). (*vol. 2024*).

Lin, Y., Ren, P., Chen, Z., Ren, Z., Yu, D., Ma, J., de Rijke, M., & Cheng, X. (2020). Meta matrix factorization for federated rating predictions. In *Proceedings of the 43rd international ACM SIGIR conference on research and development in information retrieval (SIGIR)*. https://doi.org/10.1145/3397271.3401081

Ma, X., Wen, X., Zeng, J., Xiong, P., & Han, X. (2025). KGFedRS: Knowledge graph enhanced federated recommender system. *Neural Networks*, 107840. https://doi.org/10.1016/j.neunet.2025.107840

McMahan, B., Moore, E., Ramage, D., Hampson, S., & Agüera y Arcas, B. (2017). Communication-efficient learning of deep networks from decentralized data. In *Proceedings of the 20th international conference on artificial intelligence and statistics (AISTATS)* (pp. 1273–1282).

McMahan, H. B., Ramage, D., Talwar, K., & Zhang, L. (2018). Learning differentially private recurrent language models. In *Proceedings of the 6th international conference on learning representations (ICLR)*.

Muhammad, K., Wang, Q., O'Reilly-Morgan, D., Tragos, E., Smyth, B., Hurley, N., Geraci, J., & Lawlor, A. (2020). FedFast: Going beyond average for faster training of federated recommender systems. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery and data mining (KDD)*. https://doi.org/10.1145/3394486.3403176

Perifanis, V., & Efraimidis, P. S. (2022). Federated neural collaborative filtering. *Knowledge-Based Systems*, *242*, 108441.

Qu, L., Yuan, W., Zheng, R., Cui, L., Shi, Y., & Yin, H. (2024). Towards personalized privacy: User-governed data contribution for federated recommendation. In *Proceedings of the ACM web conference 2024 (WWW)*. https://doi.org/10.1145/3589334.3645690

Rendle, S. (2010). Factorization machines. In *Proceedings of the 10th IEEE international conference on data mining (ICDM)* (pp. 995–1000). https://doi.org/10.1109/ICDM.2010.127

Rendle, S., Freudenthaler, C., Gantner, Z., & Schmidt-Thieme, L. (2009). BPR: Bayesian personalized ranking from implicit feedback. In *Proceedings of the 25th conference on uncertainty in artificial intelligence (UAI)* (pp. 452–461).

Wu, C., Wu, F., Cao, Y., Huang, Y., & Xie, X. (2021). FedGNN: Federated graph neural network for privacy-preserving recommendation. arXiv:2102.04925.

Yang, C., Yuan, W., Qu, L., & Nguyen, T. T. (2024). PDC-FRS: Privacy-preserving data contribution for federated recommender system. In *Proceedings of the 20th international conference on advanced data mining and applications (ADMA)*. https://doi.org/10.1007/978-981-96-0850-8_5

Zhang, C., Long, G., Zhou, T., Yan, P., Zhang, Z., Zhang, C., & Yang, B. (2023). Dual personalization on federated recommendation. In *Proceedings of the 32nd international joint conference on artificial intelligence (IJCAI)* (pp. 4558–4566).

Zhang, C., Long, G., Zhou, T., Zhang, Z., Yan, P., & Yang, B. (2024). GPFedRec: Graph-guided personalization for federated recommendation. In *Proceedings of the 30th ACM SIGKDD conference on knowledge discovery and data mining (KDD)*. https://doi.org/10.1145/3637528.3671702